ADVANCED NLP with spaCy

English

Chapter 2: Large-scale data analysis with spaCy

In this chapter, you'll use your new skills to extract specific information from large volumes of text. You'll learn how to make the most of spaCy's data structures, and how to effectively combine statistical and rule-based approaches for text analysis.

1 Data Structures (1)

2 Strings to hashes

3 Vocab, hashes and lexemes

4 Data Structures (2)

5 Creating a Doc

6 Docs, spans and entities from scratch

7 Data structures best practices

8 Word vectors and semantic similarity

9 Inspecting word vectors

10 Comparing similarities

11 Combining predictions and rules

12 Debugging patterns (1)

13 Debugging patterns (2)

14 Efficient phrase matching

15 Extracting countries and relationships

2 Strings to hashes

Part 1

- Look up the string “cat” in `nlp.vocab.strings` to get the hash.
- Look up the hash to get back the string.

INPUT

```
import spacy

nlp = spacy.blank("en")
doc = nlp("I have a cat")

# Look up the hash for the word "cat"
cat_hash = doc.vocab.strings["cat"]
print(cat_hash)

# Look up the cat_hash to get the string
cat_string = nlp.vocab.strings[cat_hash]
print(cat_string)
```

Run code Submit

OUTPUT

```
5439657043933447811
cat
```

5 Creating a Doc

Let's create some `Doc` objects from scratch!

Part 1

- Import the `Doc` from `spacy.tokens`.
- Create a `Doc` from the `words` and `spaces`. Don't forget to pass in the vocab!

INPUT

```
import spacy

nlp = spacy.blank("en")

# Import the Doc class
from spacy.tokens import Doc

# Desired text: "spaCy is cool!"
words = ["spaCy", "is", "cool", "!"]
spaces = [True, True, False, False]

# Create a Doc from the words and spaces
doc = Doc(nlp.vocab, words=words, spaces=spaces)
print(doc.text)
```

Run code Submit

OUTPUT

```
spaCy is cool!
```

In this exercise, you'll create the `Doc` and `Span` objects manually, and update the named entities – just like spaCy does behind the scenes. A shared `nlp` object has already been created.

- Import the `Doc` and `Span` classes from `spacy.tokens`.
- Use the `Doc` class directly to create a `doc` from the words and spaces.
- Create a `Span` for "David Bowie" from the `doc` and assign it the label `"PERSON"`.
- Overwrite the `doc.ents` with a list of one entity, the "David Bowie" `span`.

INPUT

```
import spacy

nlp = spacy.blank("en")

# Import the Doc and Span classes
from spacy.tokens import Doc, Span

words = ["I", "like", "David", "Bowie"]
spaces = [True, True, True, False]

# Create a doc from the words and spaces
doc = Doc(nlp.vocab, words=words, spaces=spaces)
print(doc.text)

# Create a span for "David Bowie" from the doc and assign it the label "PERSON"
span = Span(doc, 2, 4, label="PERSON")
print(span.text, span.label_)

# Add the span to the doc's entities
doc.ents = [span]

# Print entities' text and labels
print([(ent.text, ent.label_) for ent in doc.ents])
```

Run code Submit

OUTPUT

```
I like David Bowie
David Bowie PERSON
[('David Bowie', 'PERSON')]
```

7 Data structures best practices

The code in this example is trying to analyze a text and collect all proper nouns that are followed by a verb.

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Berlin looks like a nice city")

# Get all tokens and part-of-speech tags
token_texts = [token.text for token in doc]
pos_tags = [token.pos_ for token in doc]

for index, pos in enumerate(pos_tags):
    # Check if the current token is a proper noun
    if pos == "PROPN":
        # Check if the next token is a verb
        if pos_tags[index + 1] == "VERB":
            result = token_texts[index]
            print("Found proper noun before a verb:", result)
```

- Rewrite the code to use the native token attributes instead of lists of `token_texts` and `pos_tags`.
- Loop over each `token` in the `doc` and check the `token.pos_` attribute.
- Use `doc[token.i + 1]` to check for the next token and its `.pos_` attribute.
- If a proper noun before a verb is found, print its `token.text`.

INPUT

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Berlin looks like a nice city")

# Iterate over the tokens
for token in doc:
    # Check if the current token is a proper noun
    if token.pos_ == "PROPN":
        # Check if the next token is a verb
        if doc[token.i + 1].pos_ == "VERB":
            print("Found proper noun before a verb:", token.text)
```

Run code Submit

OUTPUT

```
Found proper noun before a verb: Berlin
```

- Use the `doc.similarity` method to compare `doc1` to `doc2` and print the result.

INPUT

```
import spacy

nlp = spacy.load("en_core_web_md")

doc1 = nlp("It's a warm summer day")
doc2 = nlp("It's sunny outside")

# Get the similarity of doc1 and doc2
similarity = doc1.similarity(doc2)
print(similarity)
```

Run code Submit

14 Efficient phrase matching

Sometimes it's more efficient to match exact strings instead of writing patterns describing the individual tokens. This is especially true for finite categories of things – like all countries of the world. We already have a list of countries, so let's use this as the basis of our information extraction script. A list of string names is available as the variable `COUNTRIES`.

- Import the `PhraseMatcher` and initialize it with the shared `vocab` as the variable `matcher`.
- Add the phrase patterns and call the matcher on the `doc`.

INPUT

```
import json
import spacy

with open("exercises/en/countries.json", encoding="utf8") as f:
    COUNTRIES = json.loads(f.read())

nlp = spacy.blank("en")
doc = nlp("Czech Republic may help Slovakia protect its airspace")

# Import the PhraseMatcher and initialize it
from spacy.matcher import PhraseMatcher

matcher = PhraseMatcher(nlp.vocab)

# Create pattern Doc objects and add them to the matcher
# This is the faster version of: [nlp(country) for country in COUNTRIES]
patterns = list(nlp.pipe(COUNTRIES))
matcher.add("COUNTRY", patterns)

# Call the matcher on the test document and print the result
matches = matcher(doc)
print([doc[start:end] for match_id, start, end in matches])
```

Run code Submit

OUTPUT

```
[Czech Republic, Slovakia]
```